GATE
CS & DA

Artificial Intelligence

Adversarial Search

Lecture No. 2

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic — Minimax Algo

Topic — α, β Pruning

Topic

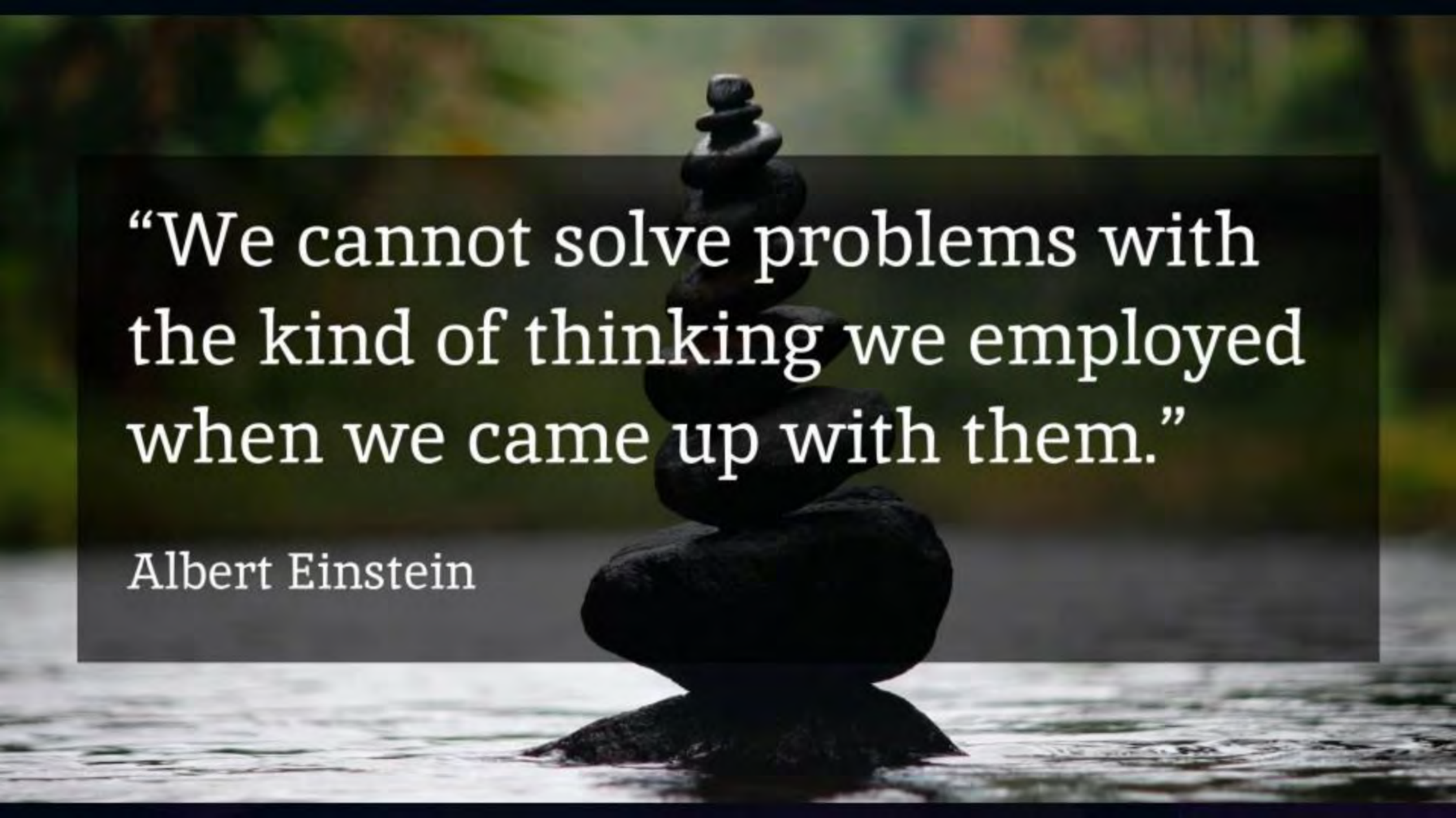
Topic

Topic

Turn on Slide map

Topics to be Covered

- Topic α, β pruning
- Topic
- Topic
- Topic
- Topic


“We cannot solve problems with the kind of thinking we employed when we came up with them.”
Albert Einstein

Adversarial Search

- Adversarial Search: A search in scenarios where multiple agents (players) have conflicting objectives, and each agent's success is often measured against the other's failure. Unlike traditional search problems (like pathfinding), adversarial search considers the presence of an opponent actively working against the agent's goal.

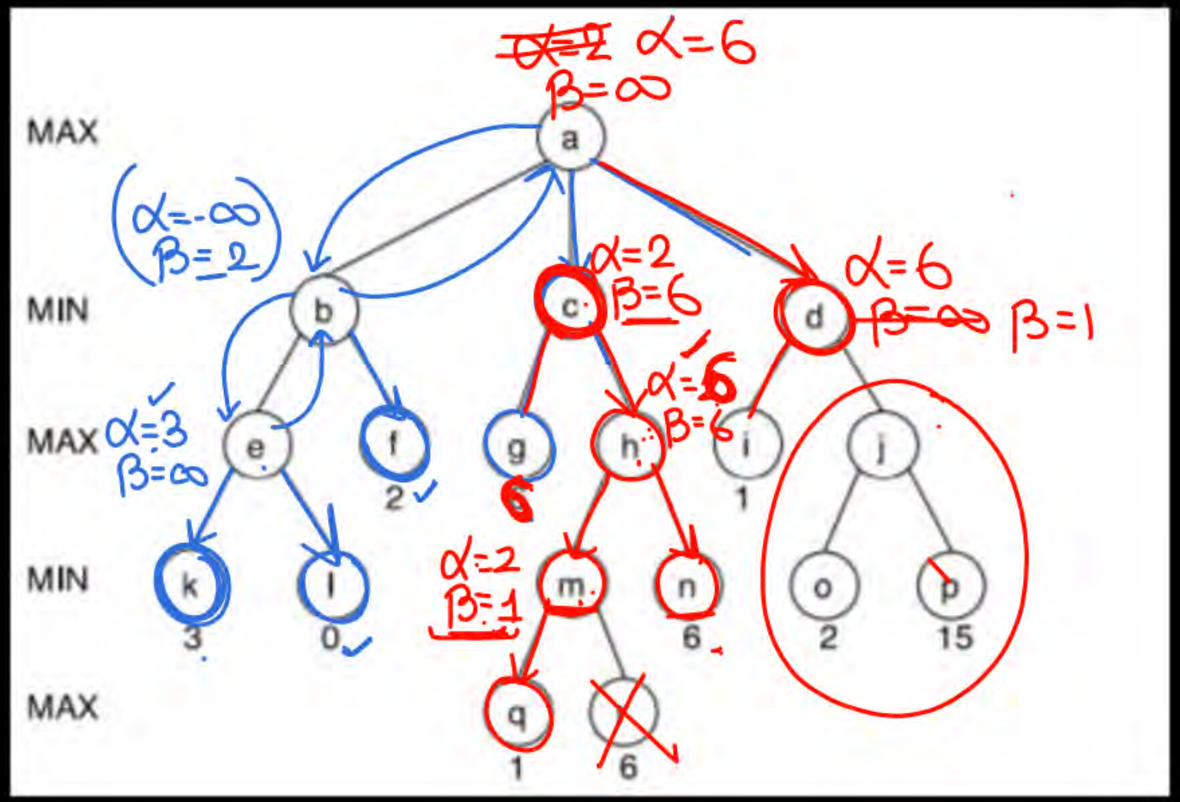
Minimax Algorithm:

- The Minimax algorithm is a recursive, decision-making algorithm used in two-player games, particularly zero-sum games where one player's gain is another player's loss. It systematically explores the game tree to determine the optimal move for a player, assuming that the opponent also plays optimally.
- It is a specialized Search Algo that returns optimal Sequence move for a player in a Zero-Sum Game.
- Recursive/ Back tracking algo which is used in decision making and Game theory and uses recursion to Search through Game Tree.
- Algo Computes minimax decision for current state.
- We have 2 players : max and min player. Max player select the maximum value and min player select the minimum value
- Depth-First Search Algo is used for exploration of Complete Game Tree.
- Used in Chess, Tic Tac Toe and other 2 player Games

Minimax Algorithm:

- Initial values for Max and Min player : + infinity and – Infinity
- This is the backtracking algorithm
- Complete and optimal solution
- Time and Space complexity
- In Minimax algorithm we use DFS

- Find the move best for Player A



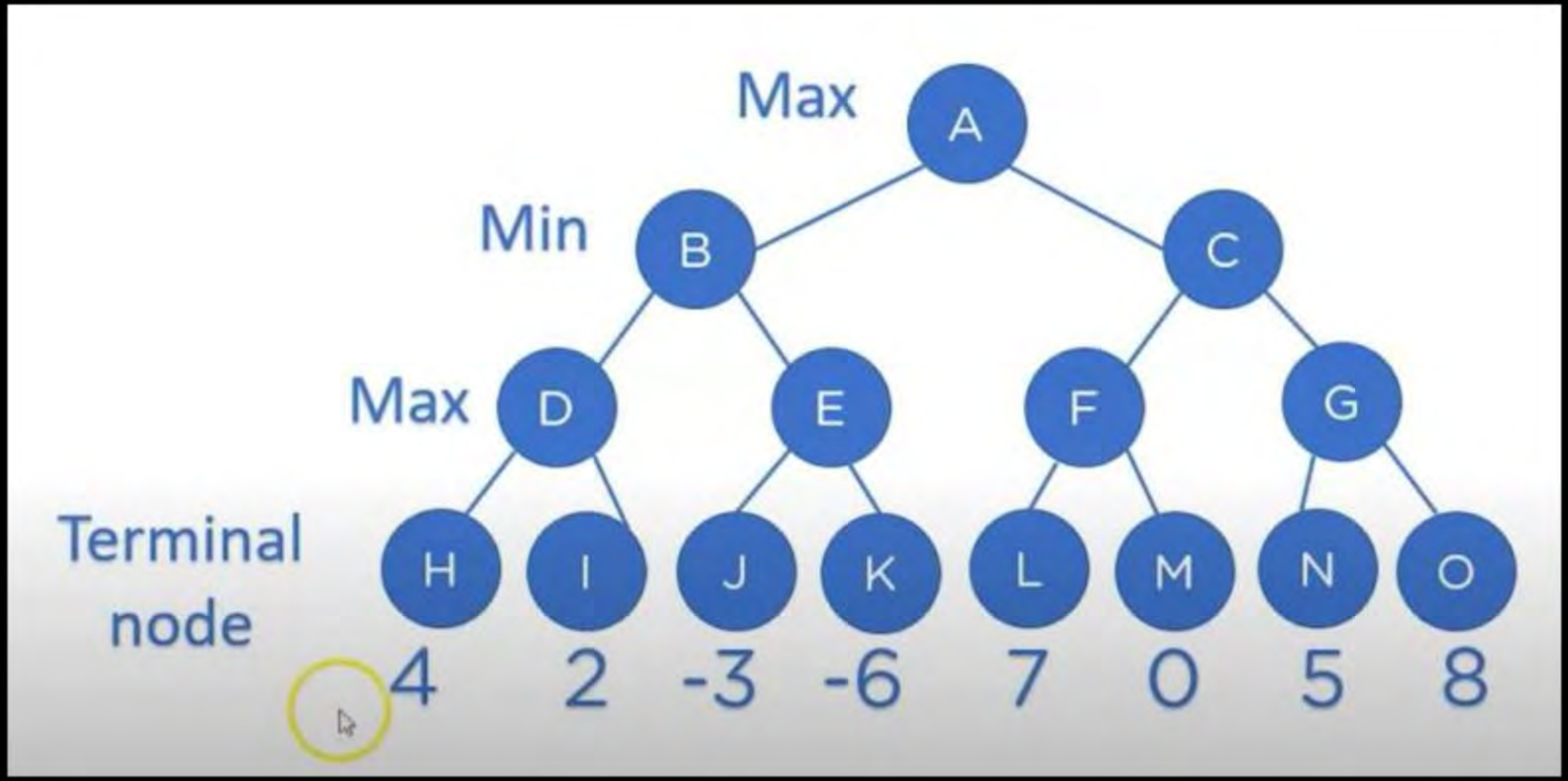
Pruned.

- Find the move best for Player A

```
          A
        / | \
      B1  B2  B3
     / \ / \ / \
    C1 C2 C3 C4 C5 C6
    |  |  |  |  |  |
    3  5  2  9  12 5
```

- Find the move best for Player A

```
          A
        / | \
     B1   B2   B3
    / \  / \  / \
  C1  C2 C3  C4 C5  C6
  |   |  |   |  |   |
  7   4  3   6  8   5
```


Max
A
Min
B
C
Max
D
E
F
G
Terminal node
H
I
J
K
L
M
N
O
4
2
-3
-6
7
0
5
8

Minimax Algorithm:

- Conceptual Overview Two Players:
  - Often referred to as the "Maximizer" and the "Minimizer.
  - "Maximizer: Tries to maximize their score.
  - Minimizer: Tries to minimize the Maximizer's score (which is equivalent to maximizing their own score if you flip the sign).
- Game Tree: A tree structure representing all possible moves (edges) from a given game state (nodes).
- Root Node: Represents the current state of the game.
- Leaf Nodes: Represent terminal states where the game ends (e.g., win, loss, or draw).

Minimax Algorithm:

- Minimax Principle
- The algorithm assumes that both players play optimally.
- It simulates all possible moves from the current state down to terminal states, then back-propagates the values up the tree to make the best decision at the root.

Minimax Algorithm:

- Minimax Algorithm Steps
- Generate the Game Tree: Start from the root node (current game state) and recursively expand all possible moves down to a certain depth or until terminal nodes (end of the game) are reached.
- Evaluate Terminal Nodes:
  - At the terminal nodes (leaf nodes), assign values based on the outcome:
  - Positive value (e.g., +1): A win for the Maximizer.
  - Negative value (e.g., -1): A win for the Minimizer.
  - Zero (0): A draw.

Minimax Algorithm:

- Minimax Algorithm Steps
- Backpropagate the Values:
  - Backpropagate the values from the terminal nodes up to the root.
  - At Maximizer nodes (Max nodes), select the maximum value from the child nodes.
  - At Minimizer nodes (Min nodes), select the minimum value from the child nodes.
  - Select the Best Move: At the root node (current game state), the Maximizer chooses the move corresponding to the child node with the highest value.

Alpha-Beta Pruning

- Alpha-beta pruning is an optimization technique for the minimax algorithm, commonly used in decision-making for two-player games like chess, tic-tac-toe, and other adversarial games. The main purpose of alpha-beta pruning is to reduce the number of nodes evaluated by the minimax algorithm, thereby speeding up the decision-making process without affecting the final decision.

Alpha-Beta Pruning

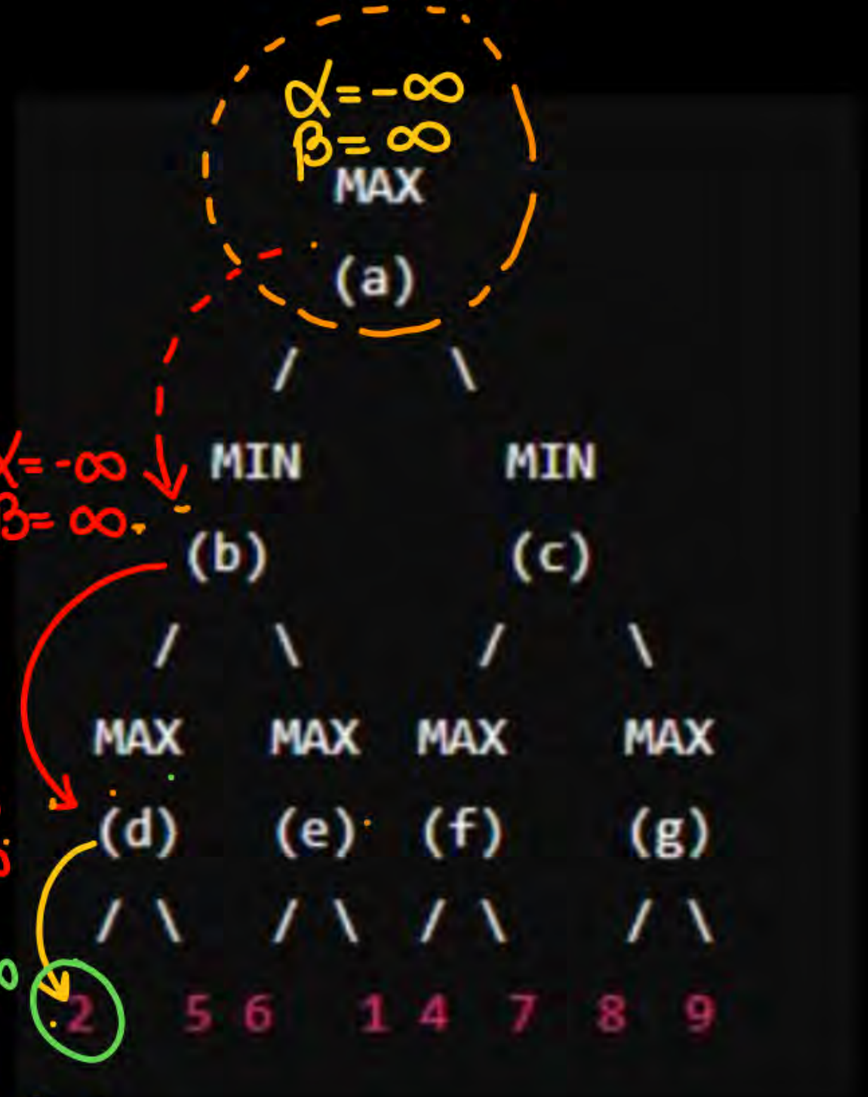
- This is advanced version of minimax algorithm
- Because minimax algorithm has to explore all the nodes thus time complexity was _____
- But in this new algorithm we explore less number of nodes
- Alpha : is for max player
- Beta : is for min player

Procedure for α, β pruning

- Algo : DFS
- α : Max player changes it New α = Max(Present α, Childnodes)
  - → Min player donot change it
- β : Min player changes it , New β = Min(Present β, Childnodes)
- Initial value of $\alpha = -\infty$, $\beta = \infty$,
- Pruning :- if value of $\alpha \geq \beta$ at any node.

Initialise

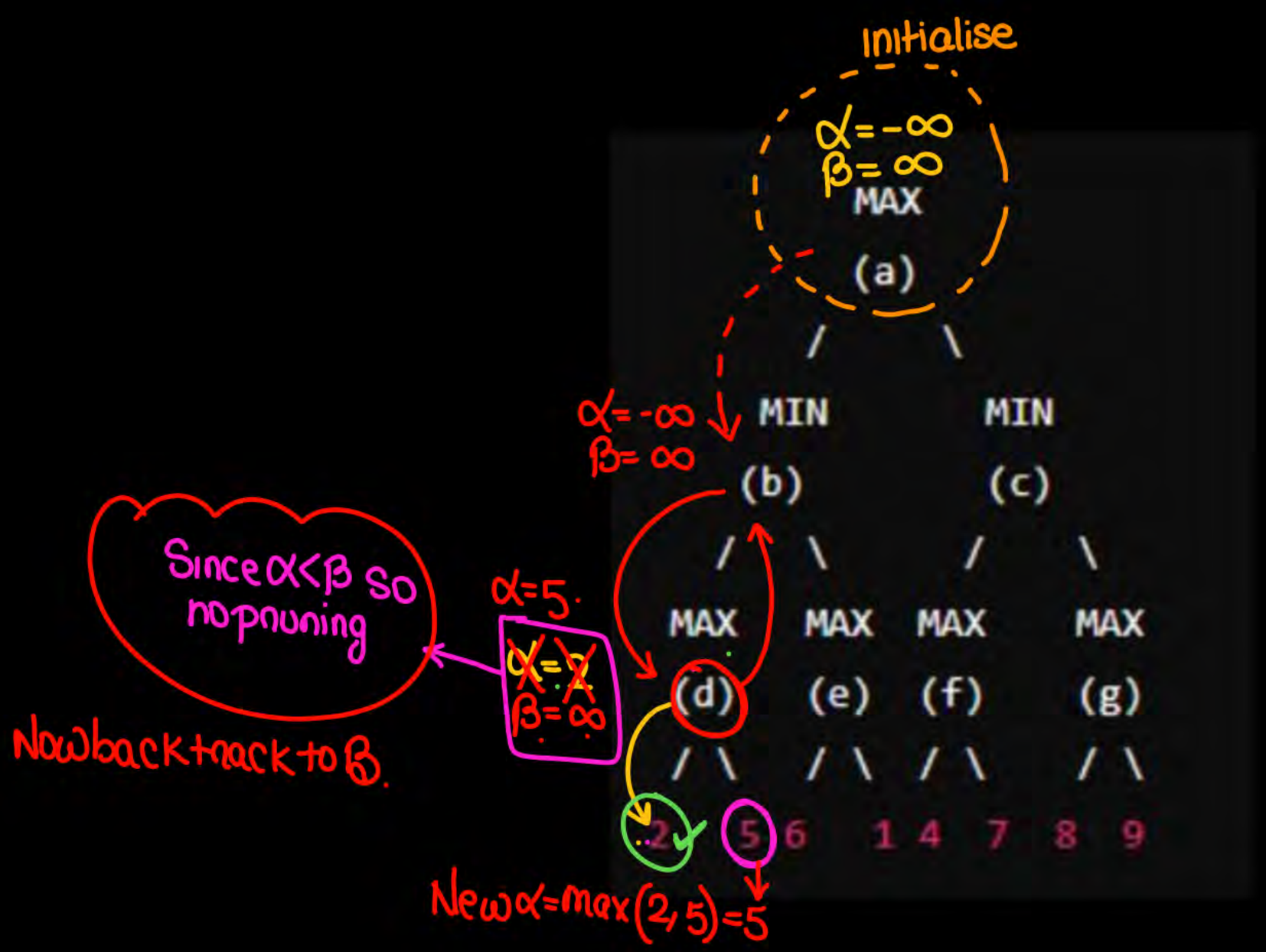
$\alpha = -\infty$
$\beta = \infty$



$\alpha = -\infty$
$\beta = \infty$

$\rightarrow \alpha = -\infty$
$\beta = \infty$

Initial value of $\alpha = -\infty$
Child node $\Rightarrow +2$
New $\alpha = \max(-\infty, 2) = 2$


Initialise
α=-∞
β=∞
MAX
(a)
α=-∞
β=∞
MIN
MIN
(b)
(c)
MAX
MAX
MAX
MAX
(d)
(e)
(f)
(g)
2
5
6
1
4
7
8
9
α=5.
α=α
β=∞
Since α<β so no pruning
Now back track to β.
New α=max(2,5)=5

d is child of b

⇒ Value of child 'd' ⇒ 5

⇒ Value of child 'd' ⇒ given by $\alpha = 5$

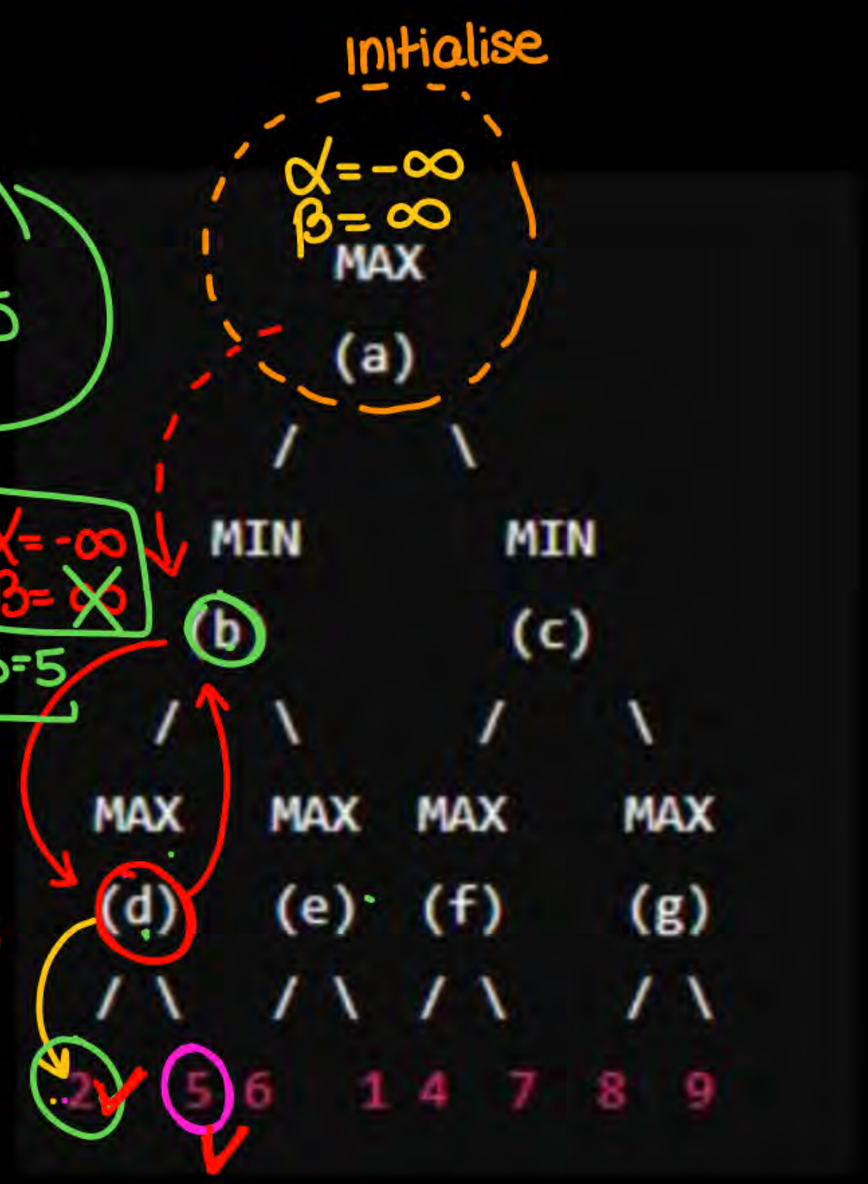
Now update β

New $\beta = \min(\infty, 5)$

$\beta = 5$

$\alpha = -\infty, \beta = 5$

(Now from $\beta \to E$)



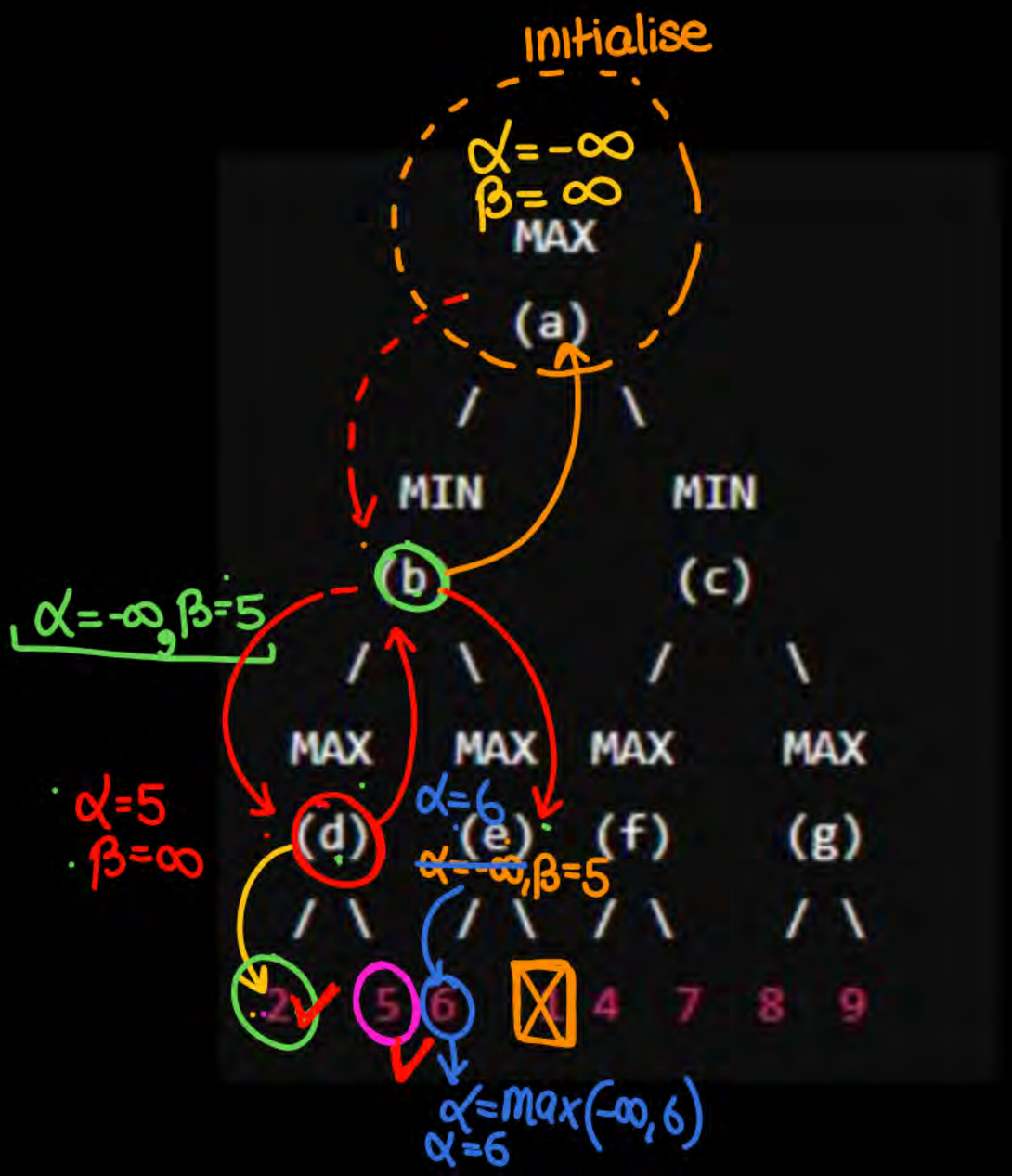
(Now $\beta \to E$
info of α, β is also given)

So @ e $(\alpha = 6, \beta = 5)$
$(\alpha \geq \beta)$

$\boxed{\text{So 1 is Pruned}}$

So as e is updated
β of b $\Rightarrow \min(5,6)$
$\Rightarrow 5$

Now back track to a



@ a

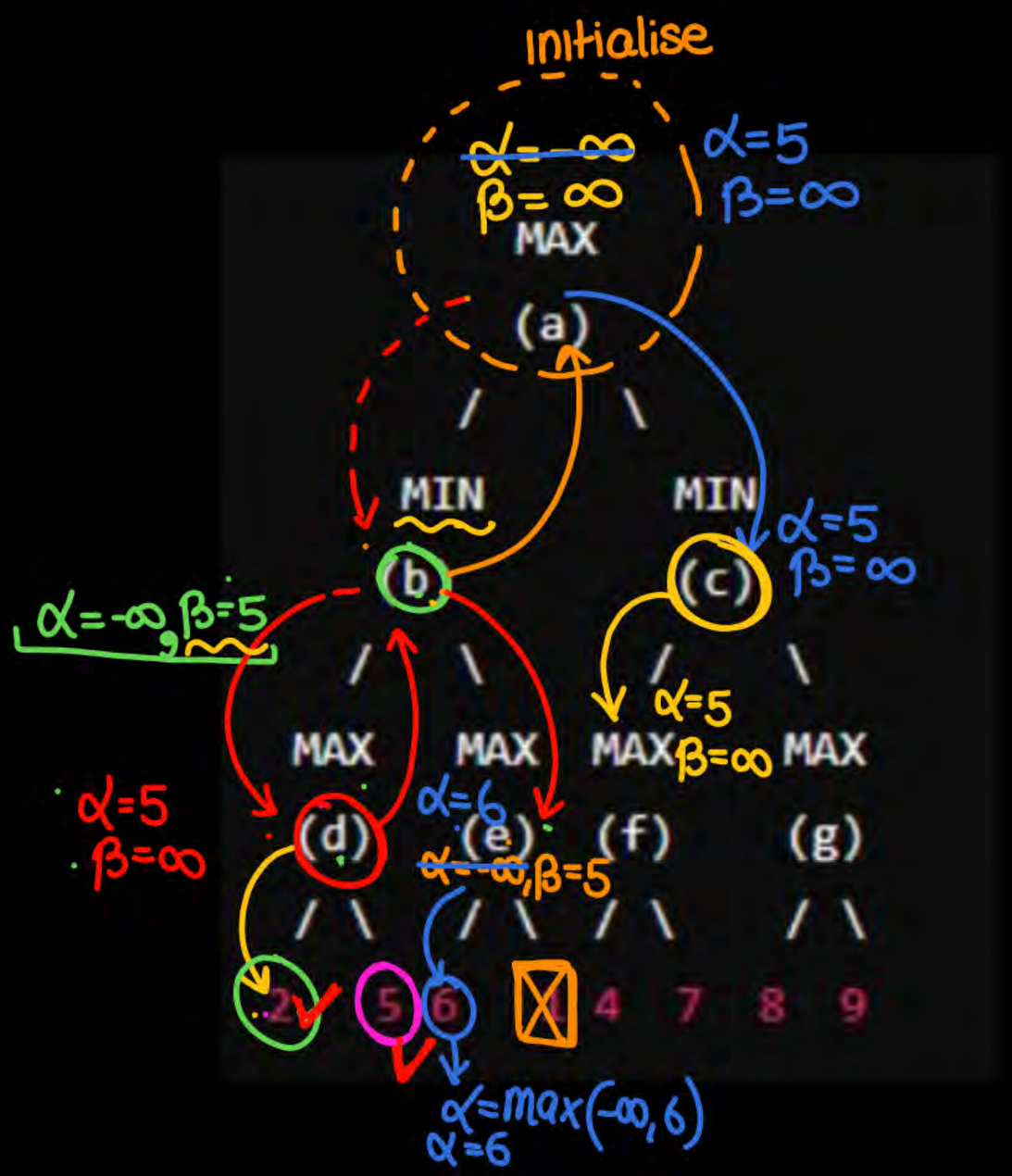
$\alpha = \text{Max}(-\infty, 5)$

$\alpha = 5$

Now move to C $(\alpha=5, \beta=\infty)$

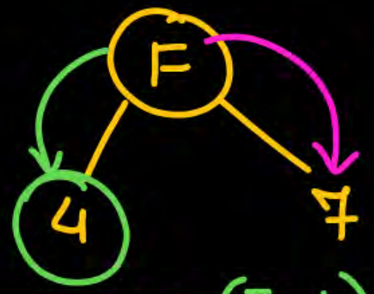
Now move to F $(\alpha=5, \beta=\infty)$

F will update α



@ F

$\left(\begin{matrix}\alpha=5\\ \beta=\infty\end{matrix}\right) \Rightarrow$ No pruning



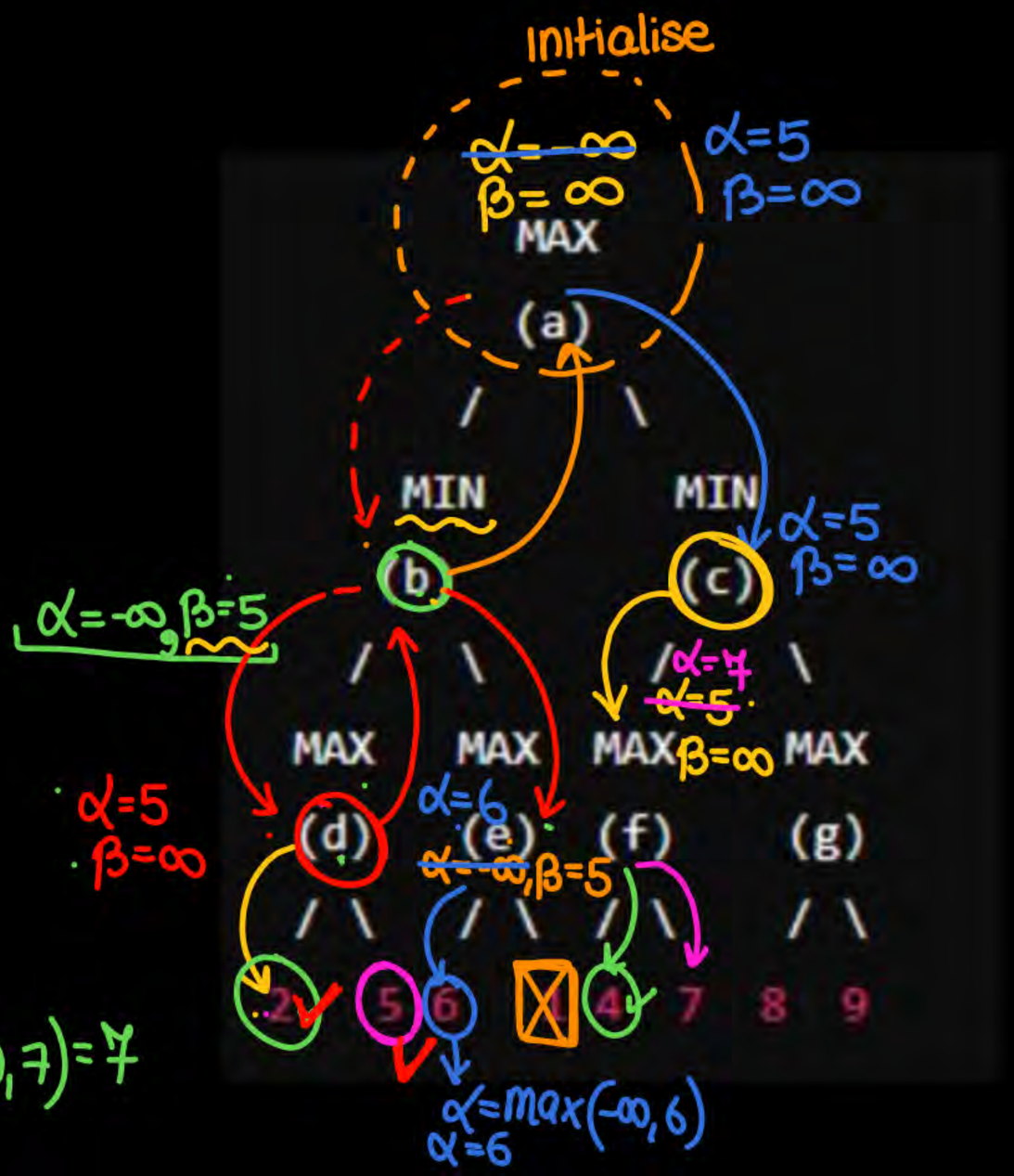
new $\alpha = \max(5,4)$
$= 5$

new $\alpha = \max(5,7) = 7$

Now @ F $\begin{matrix}\alpha=7\\ \beta=\infty\end{matrix}$

Now Backtrack to C

@C $\Rightarrow$ new $\beta = \min(\infty,7) = 7$



@ C $\beta=7, \alpha=5$

no pruning.

from C → G

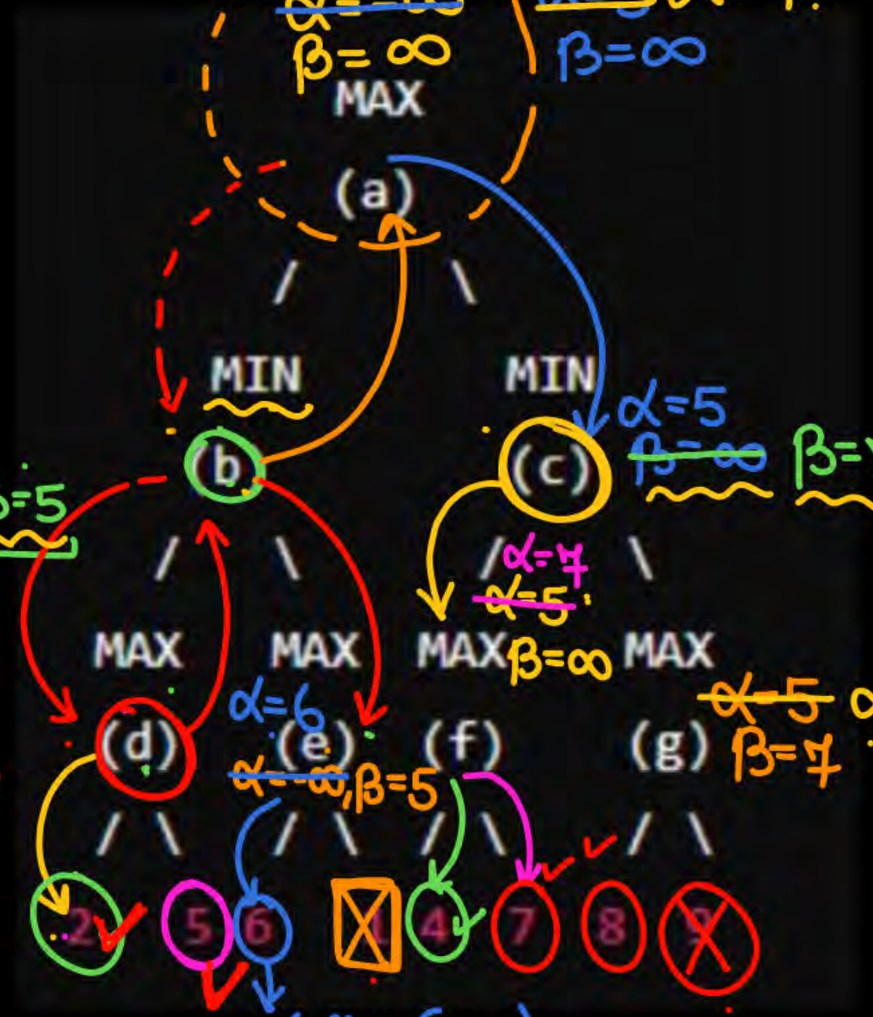
@ G

new $\alpha = \max(5,8)$

$= 8$

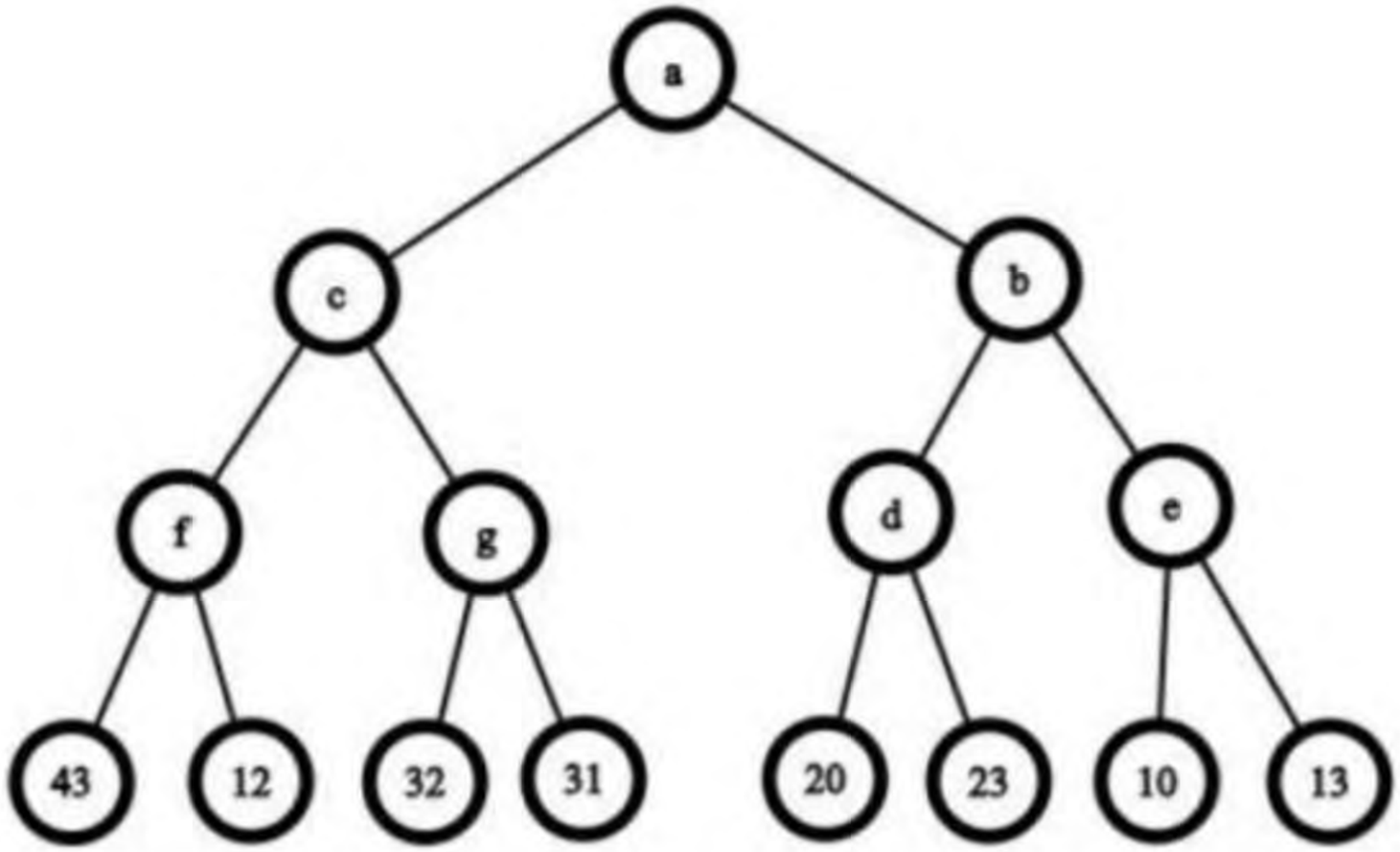
So $\alpha = 8$

So Pruning @ 1, 9

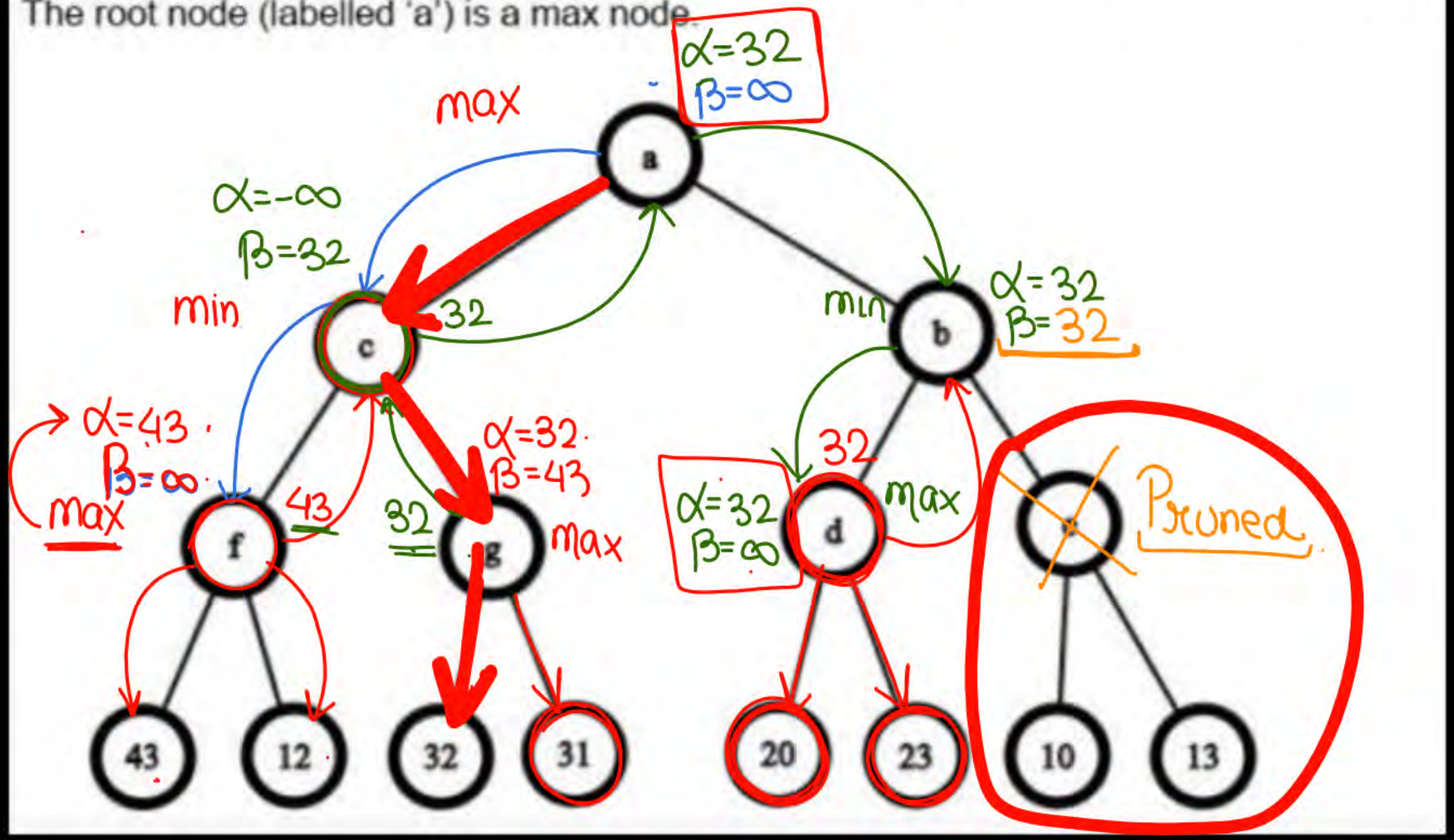


```
          MAX
          (A)
         /   \
      MIN     MIN
      (B)     (C)
     /  \     /  \
   MAX  MAX  MAX  MAX
  (D)  (E)  (F)  (G)
  / \  / \  / \  / \
 1   4 5  2 6  3 7  9
```

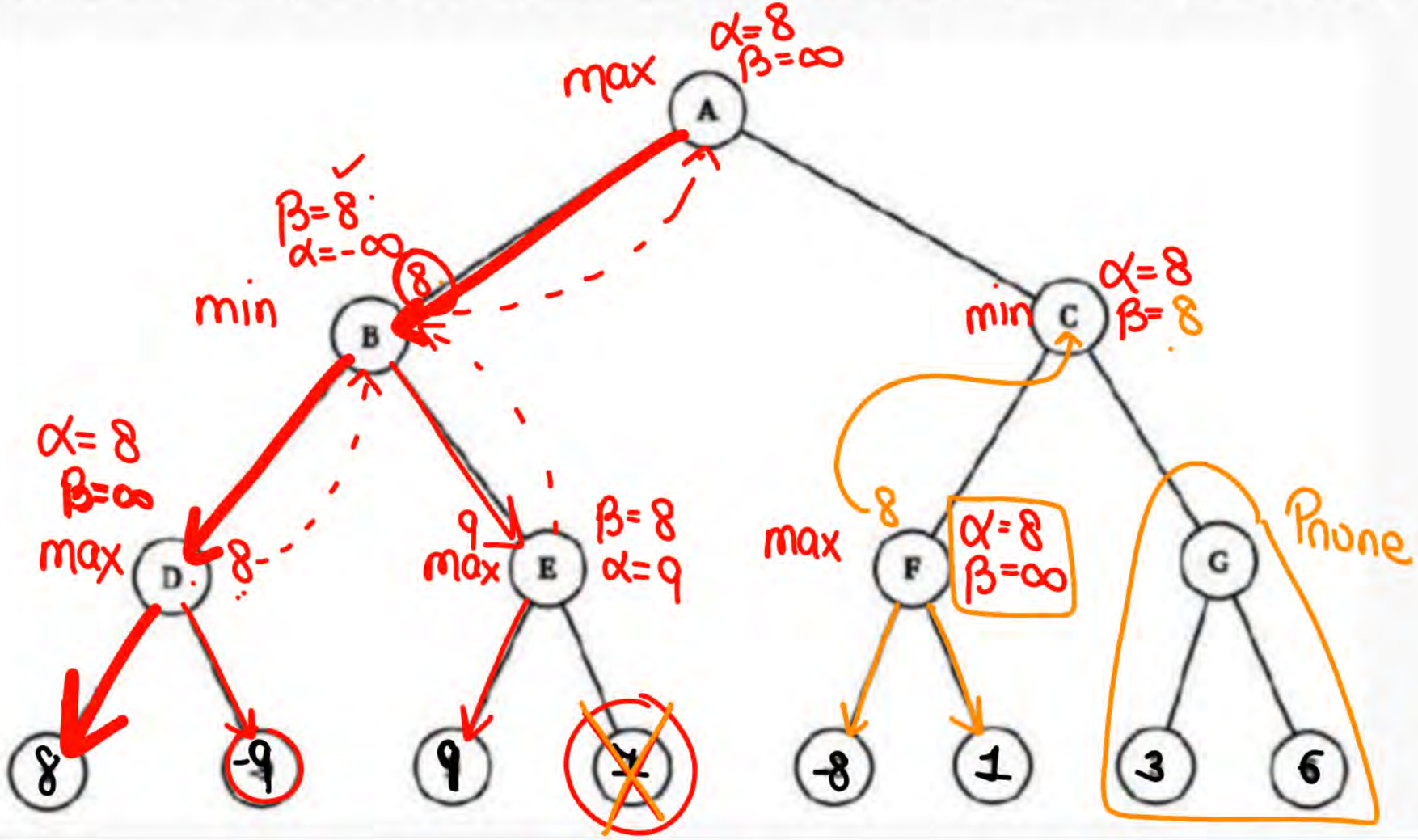
What is the value of the root node after MiniMax search over the given tree? The root node (labelled 'a') is a max node.



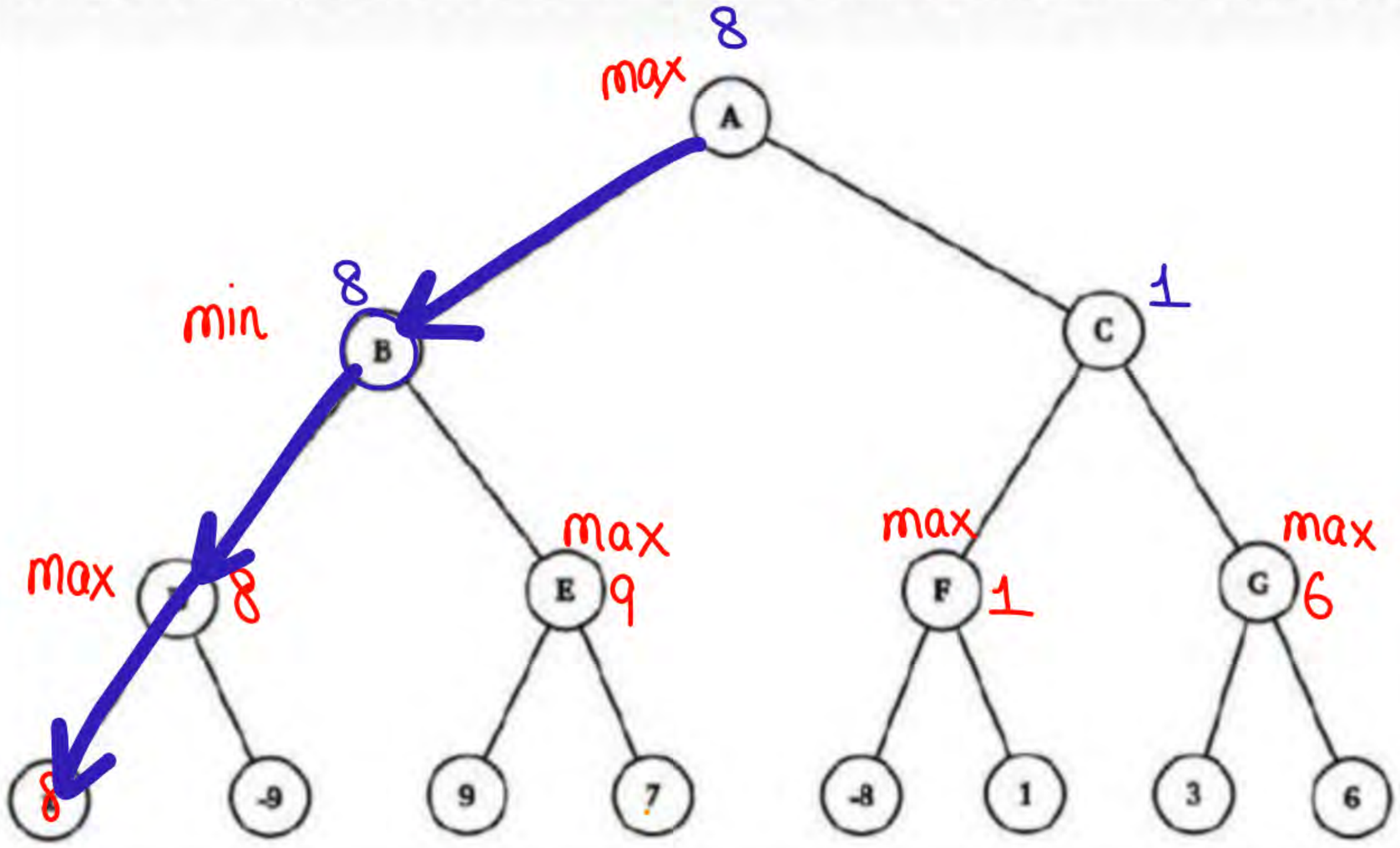
How many nodes (including leaves and internal nodes) will an alpha-beta pruning based MiniMax search visit in the given tree, assuming each successor is ordered left to right? The root node (labelled 'a') is a max node.



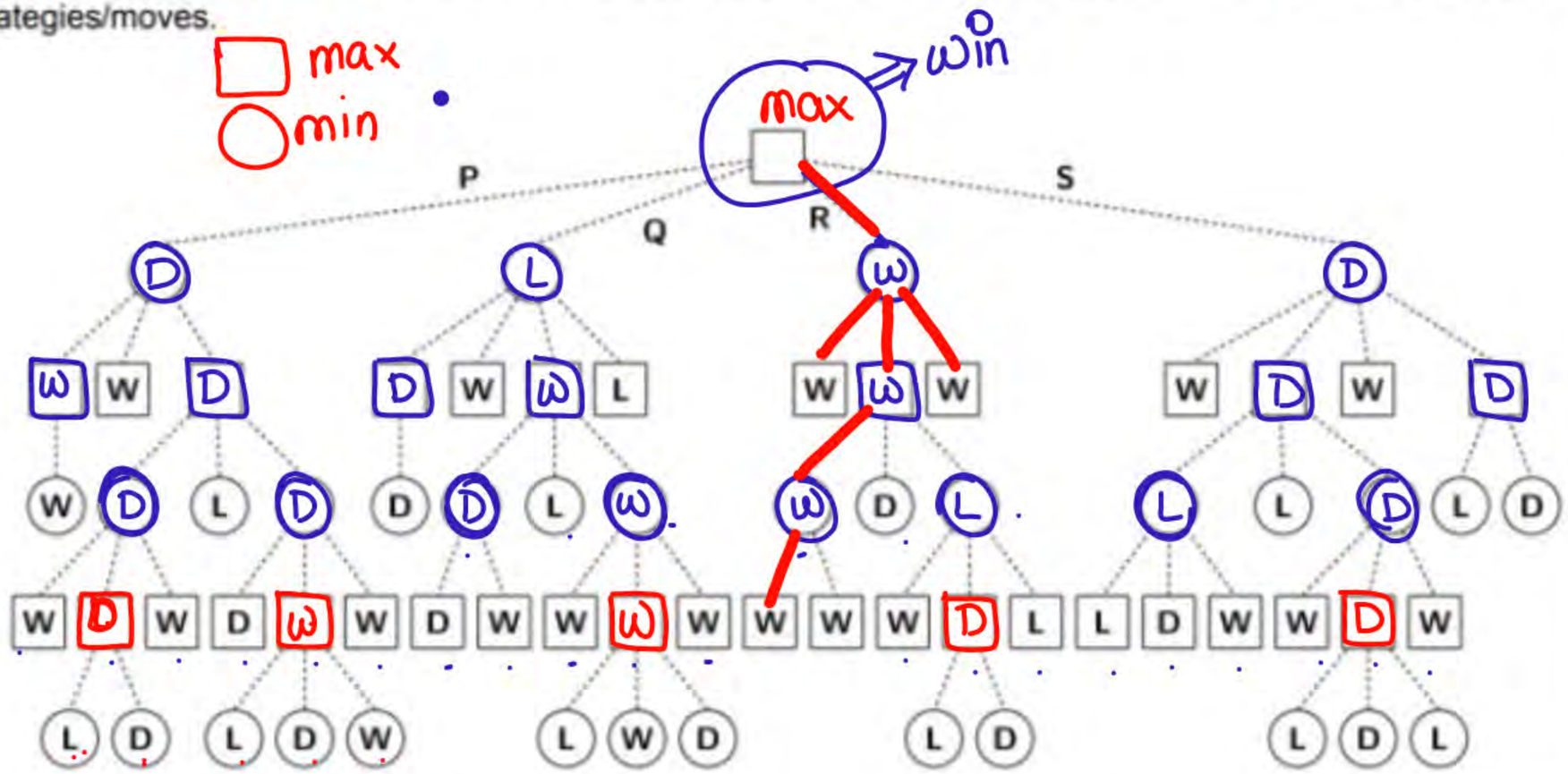
7) Player 1 (max node) chooses the left action for which of the following value(s) of x. Assume optimal play and Player 1's turn at Node A.



7) Player 1 (max node) chooses the left action for which of the following value(s) of x. Assume optimal play and Player 1's turn at Node A.



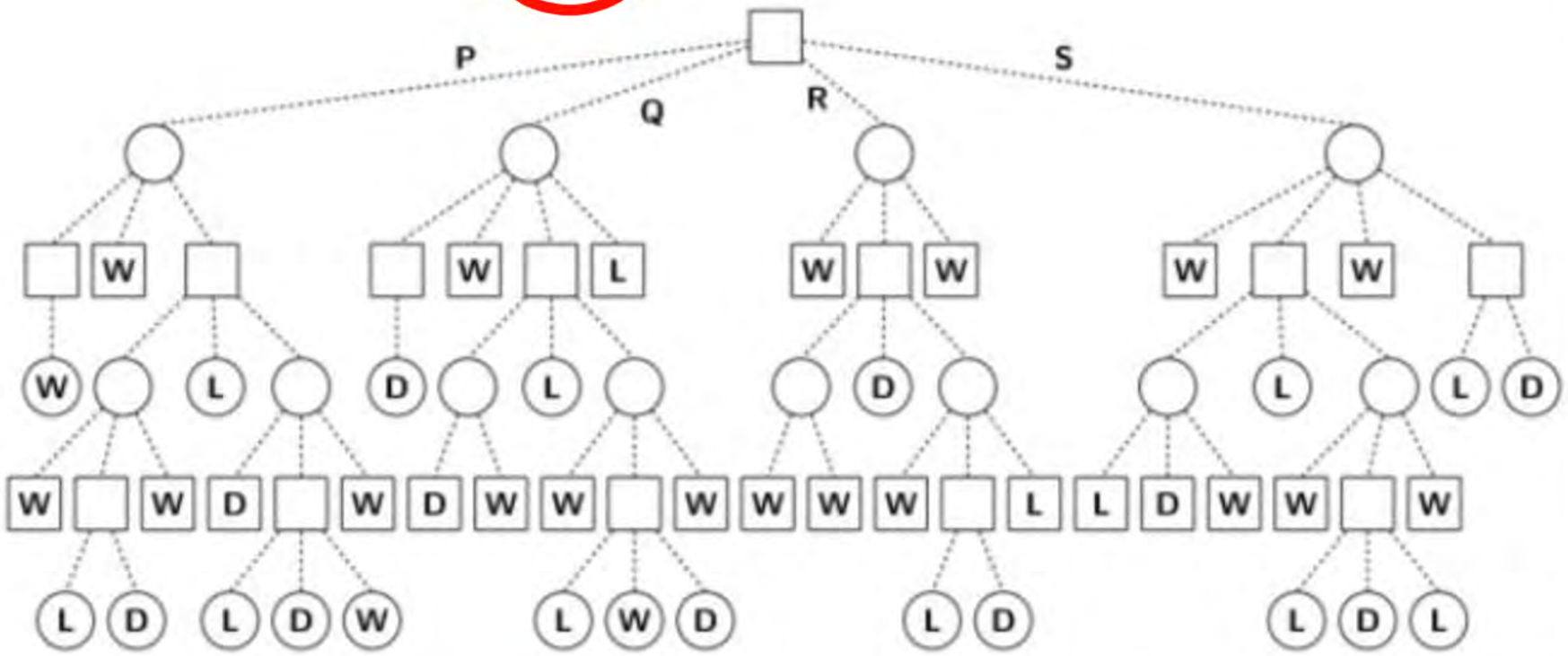
The figure shows a game tree with evaluations W (win), L (loss) and D (draw) from Max's perspective. In this game tree the labels P, Q, R, S indicate strategies/moves.



What is the outcome (W,L or D) of the game when both players play perfectly?

The figure shows a game tree with evaluations W (win), L (loss) and D (draw) from Max's perspective. In this game tree the labels P, Q, R, S indicate strategies/moves.

R

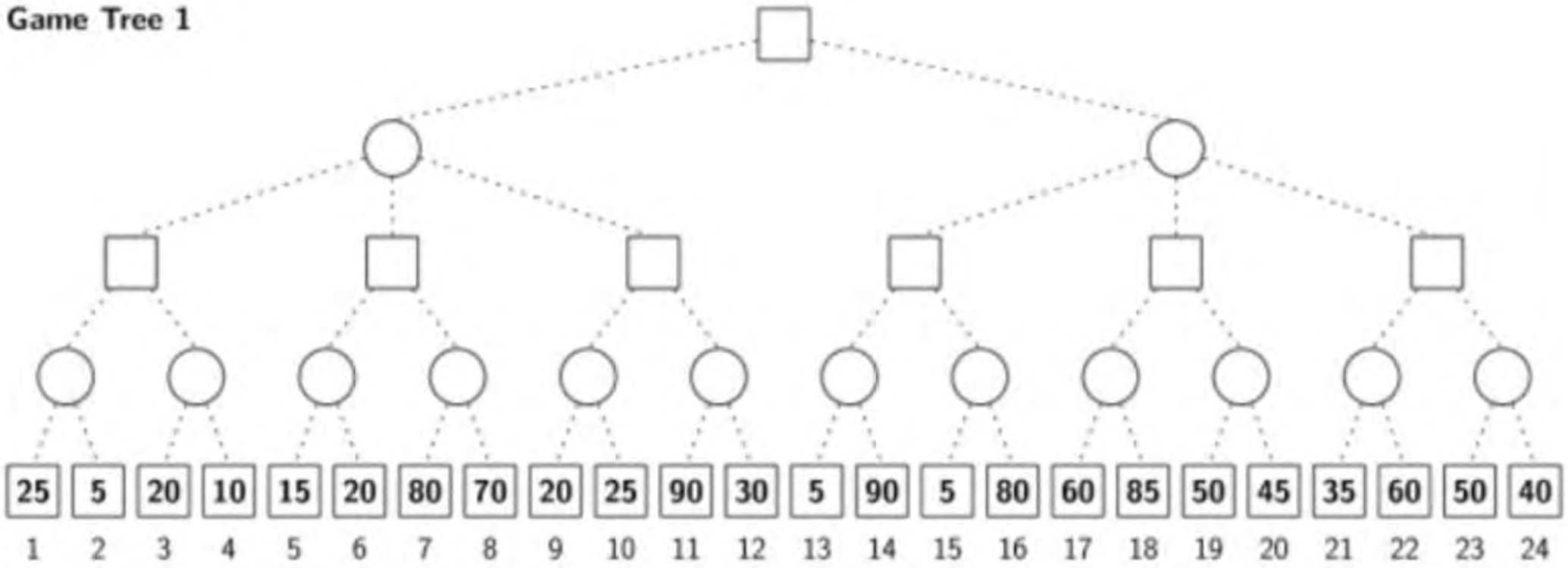


Which of the moves P,Q,R,S are best moves for Max?

9) What is the MinMax value of the game?

The figure shows a 4-ply game tree with evaluation function values at the horizon. The nodes in the horizon carry reference numbers (sequence numbers going left to right) at the bottom. Use these reference numbers when you want to enter a list of nodes. Where necessary assume top-down and left-to-right node order. Use this game tree to answer the following questions.

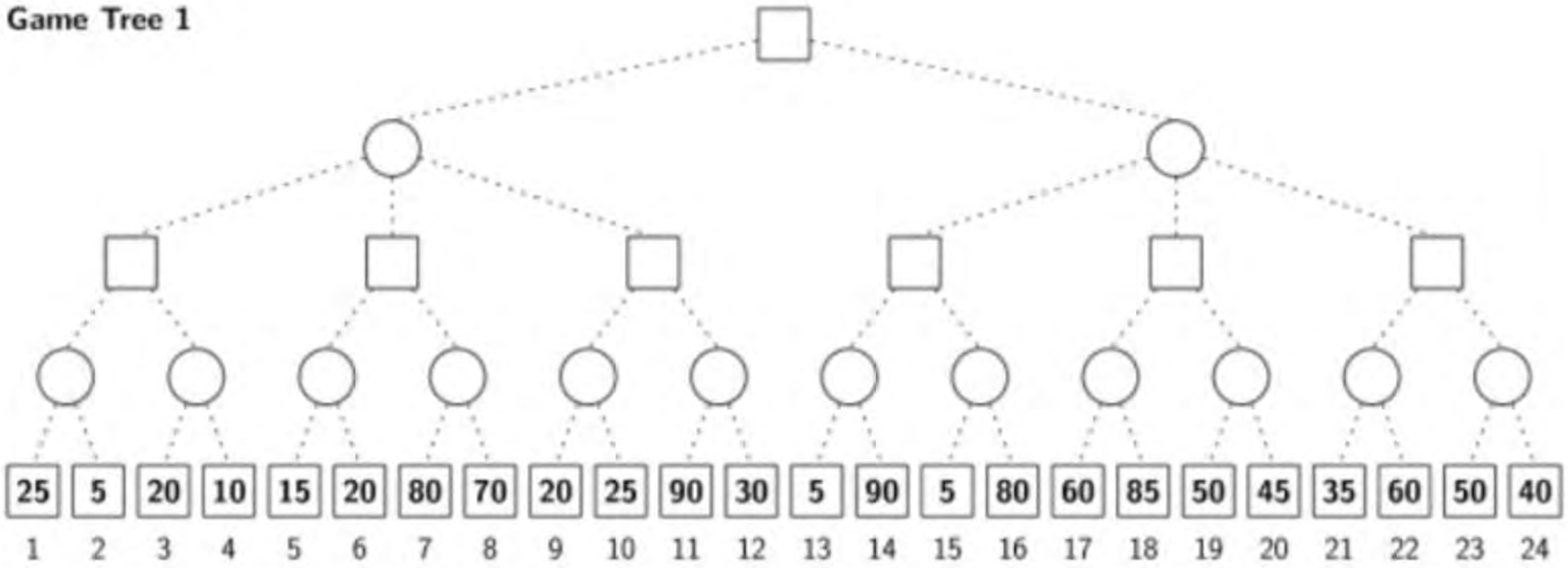
Game Tree 1



10) List the nodes (node reference numbers) in the best strategy.

The figure shows a 4-ply game tree with evaluation function values at the horizon. The nodes in the horizon carry reference numbers (sequence numbers going left to right) at the bottom. Use these reference numbers when you want to enter a list of nodes. Where necessary assume top-down and left-to-right node order. Use this game tree to answer the following questions.

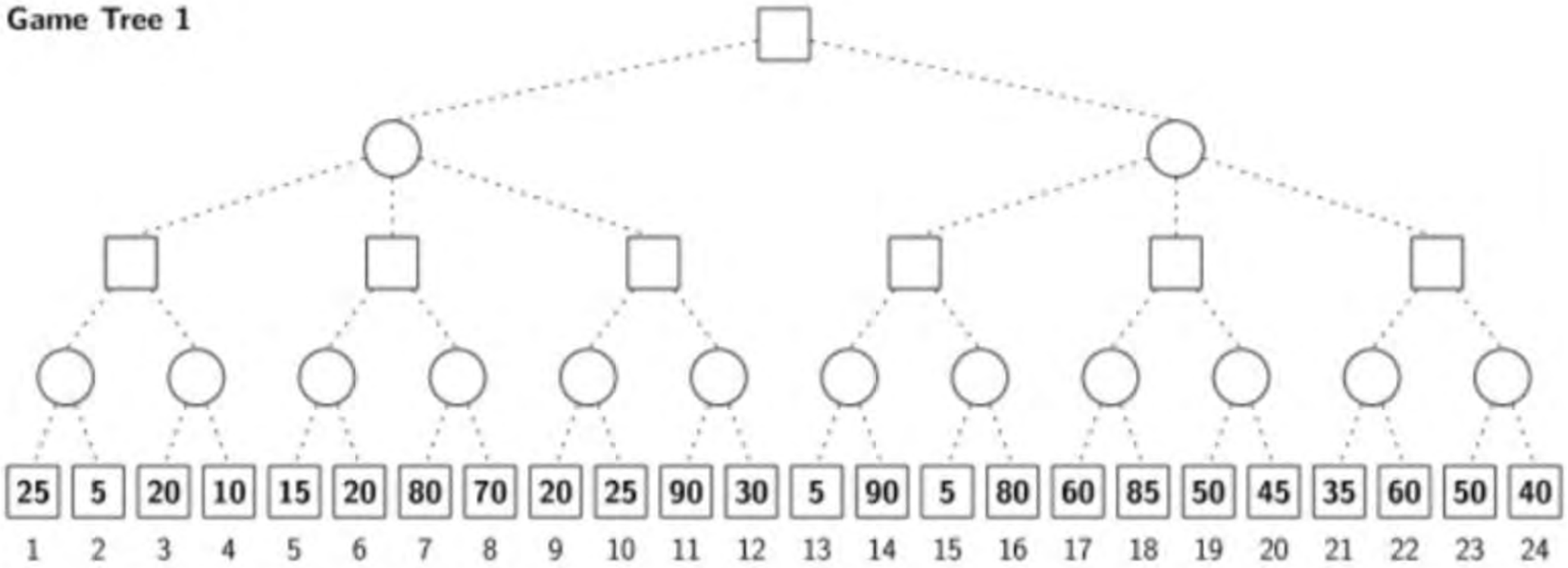
Game Tree 1



12) How many nodes are pruned (unexplored) by AlphaBeta algorithm? Count only the horizon nodes.

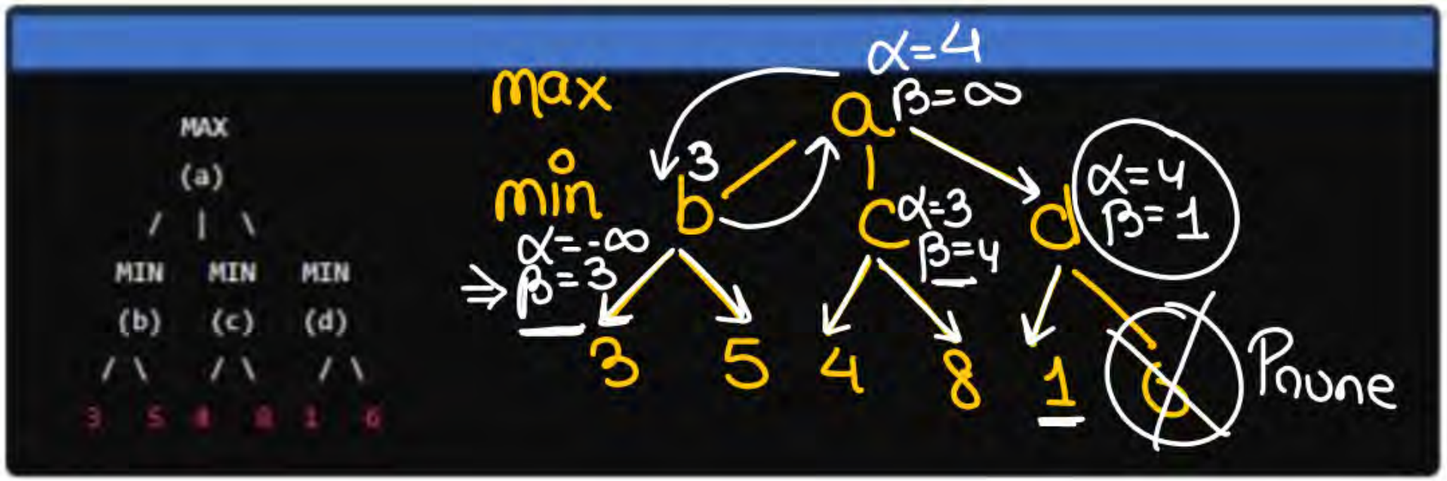
The figure shows a 4-ply game tree with evaluation function values at the horizon. The nodes in the horizon carry reference numbers (sequence numbers going left to right) at the bottom. Use these reference numbers when you want to enter a list of nodes. Where necessary assume top-down and left-to-right node order. Use this game tree to answer the following questions.

Game Tree 1



Consider the following game tree. The value below each node is the output of the utility function.

The subtrees rooted at which of these nodes will be pruned because of alpha-beta pruning?



Options:

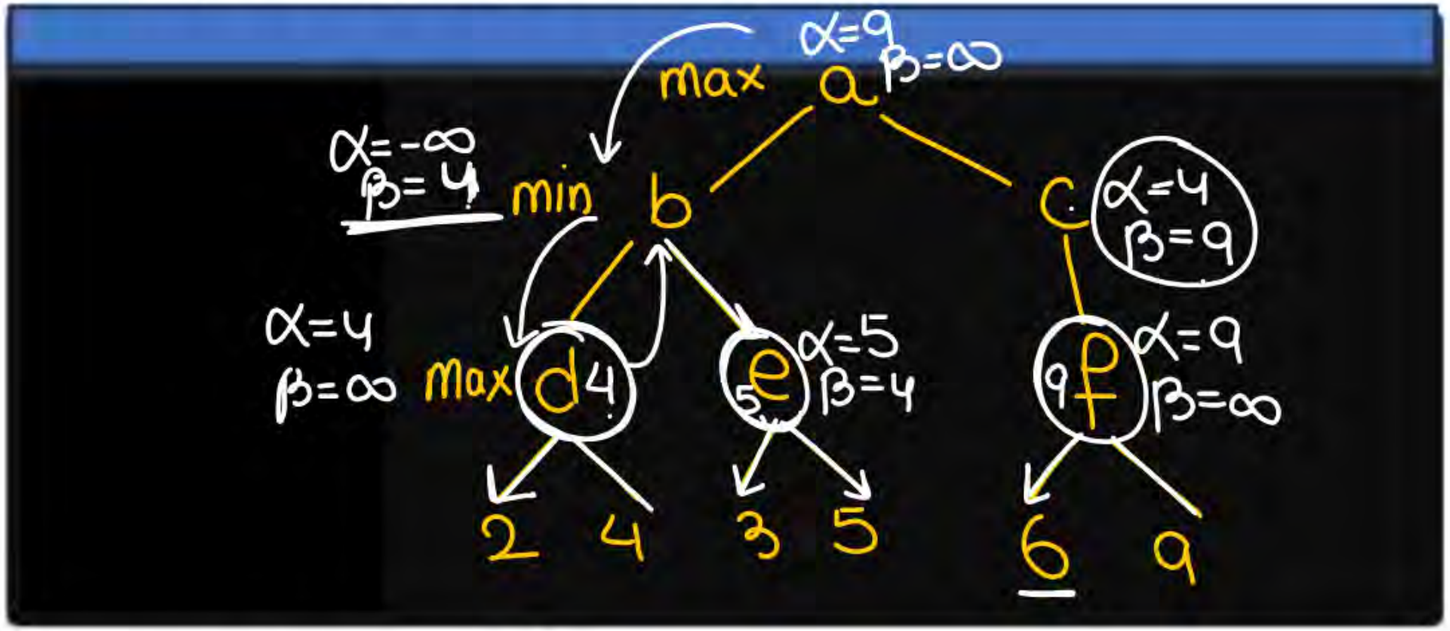
(A) Nodes under (d)

(B) Nodes under (b)

(C) Nodes under (c)

(D) Nodes under (d) and (c)

Consider this game tree. The value below each node is the output of the utility function. Which nodes will be pruned during the alpha-beta pruning process?



No pruning final α=9.

Consider this game tree. The value below each node is the output of the utility function. Which nodes will be pruned during the alpha-beta pruning process?

```
        MAX
         (a)
       /    \
     MIN     MIN
     (b)     (c)
    /   \   /   \
   MAX   MAX    MAX
  (d)   (e)     (f)
 / \   / \     / \
2   4 3   5   6   9
```

Options:

(A) Subtrees under node `(e)`

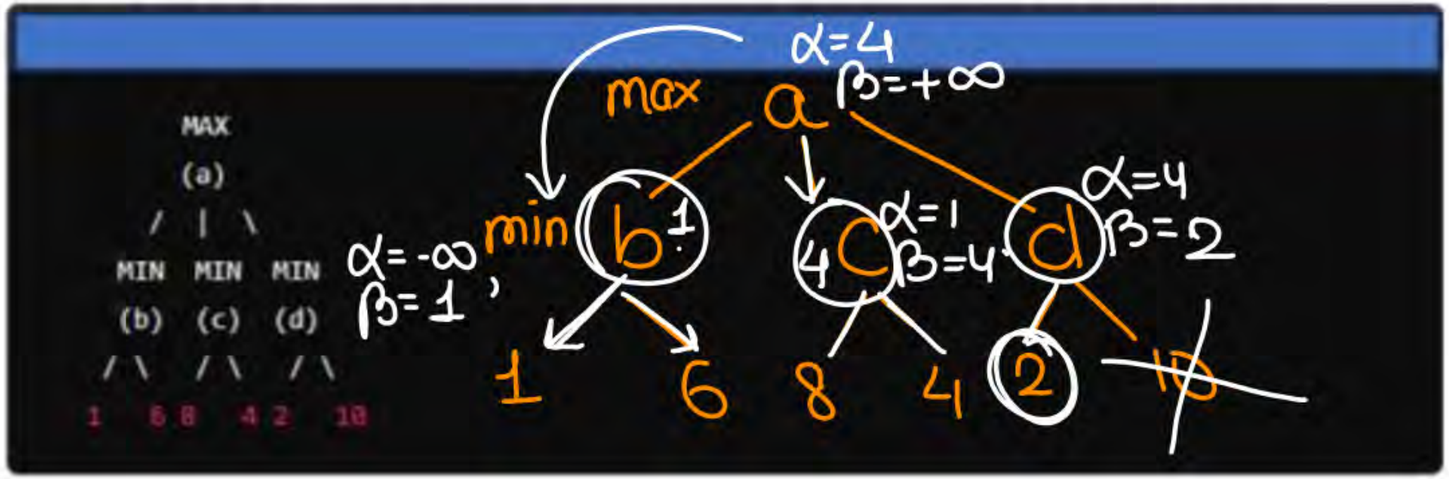
(B) Subtrees under node `(f)`

(C) Subtrees under node `(b)`

(D) Subtrees under node `(d)` and `(e)`

Q3.

For the game tree below, which of the subtrees will be pruned due to alpha-beta pruning?



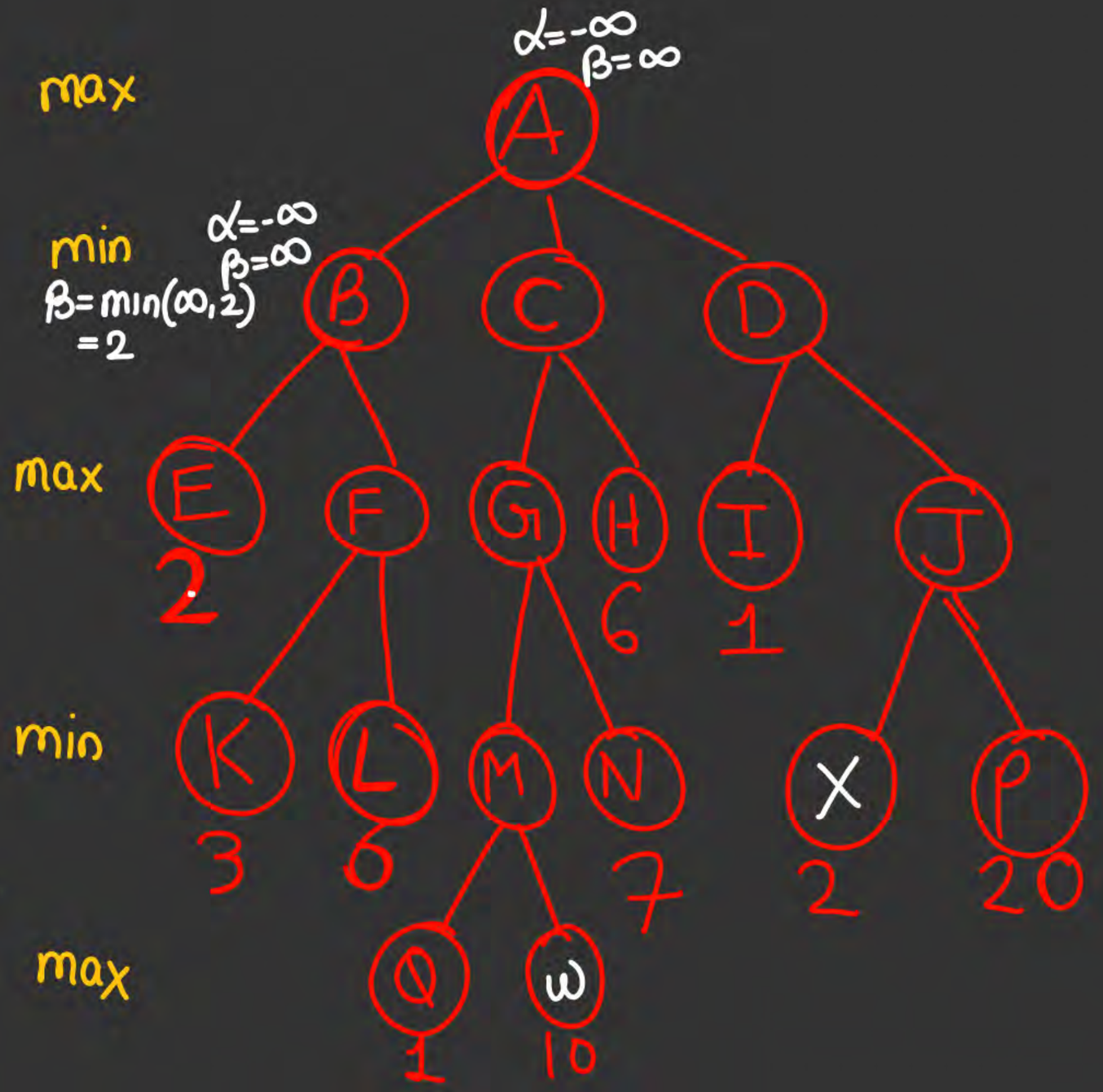
Options:

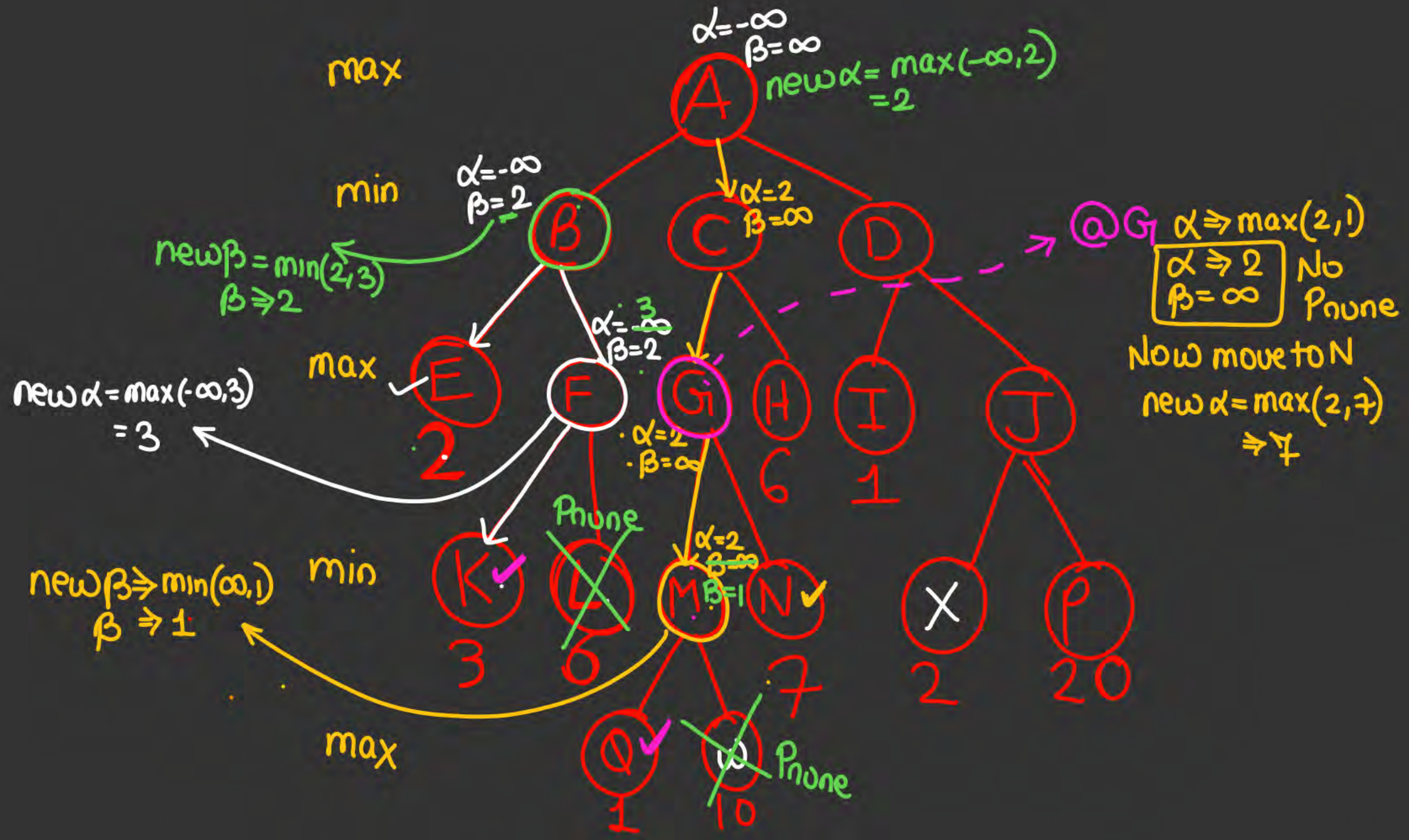
(A) Nodes under (b)

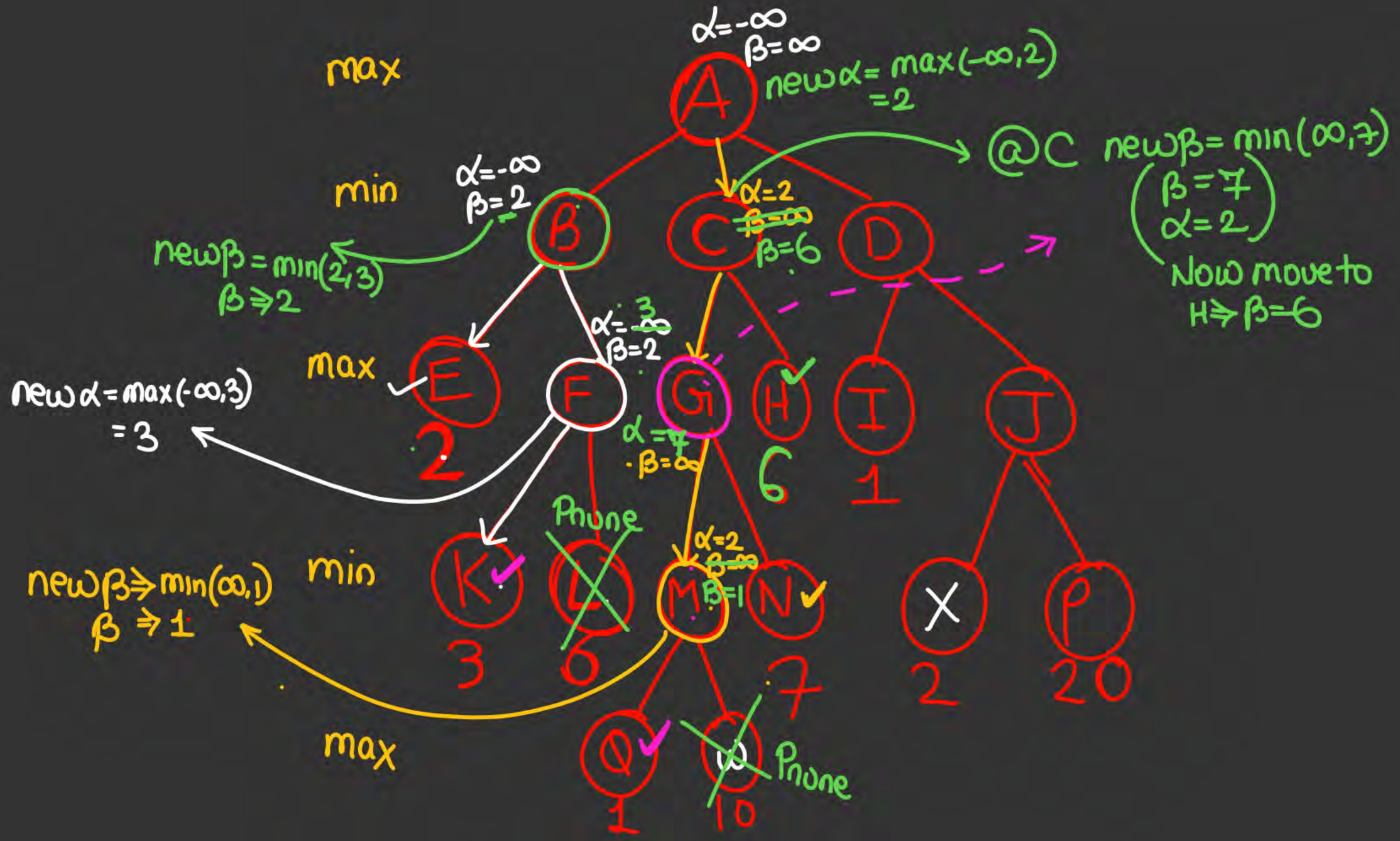
(B) Nodes under (c) and (d)

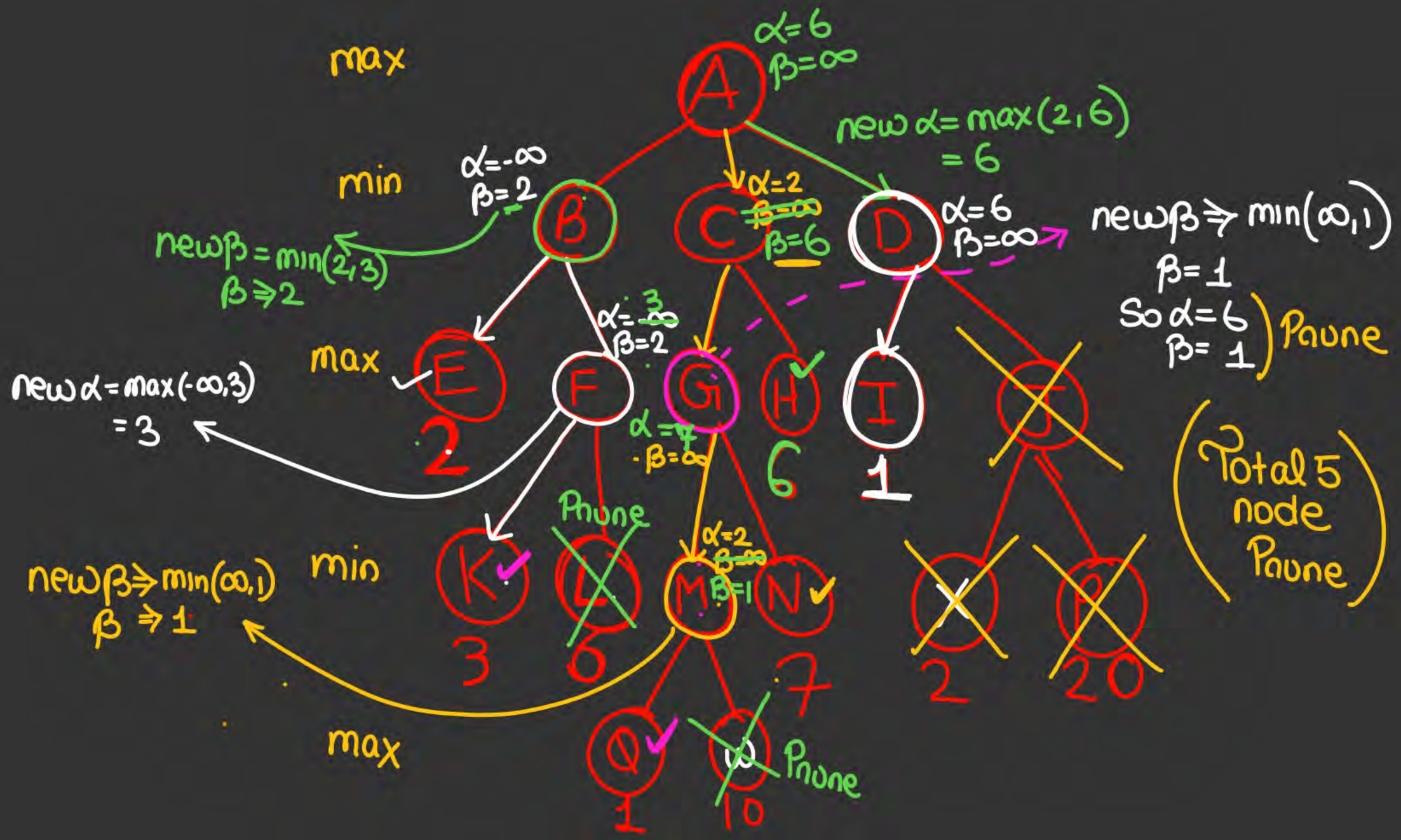
(C) Nodes under (d) only

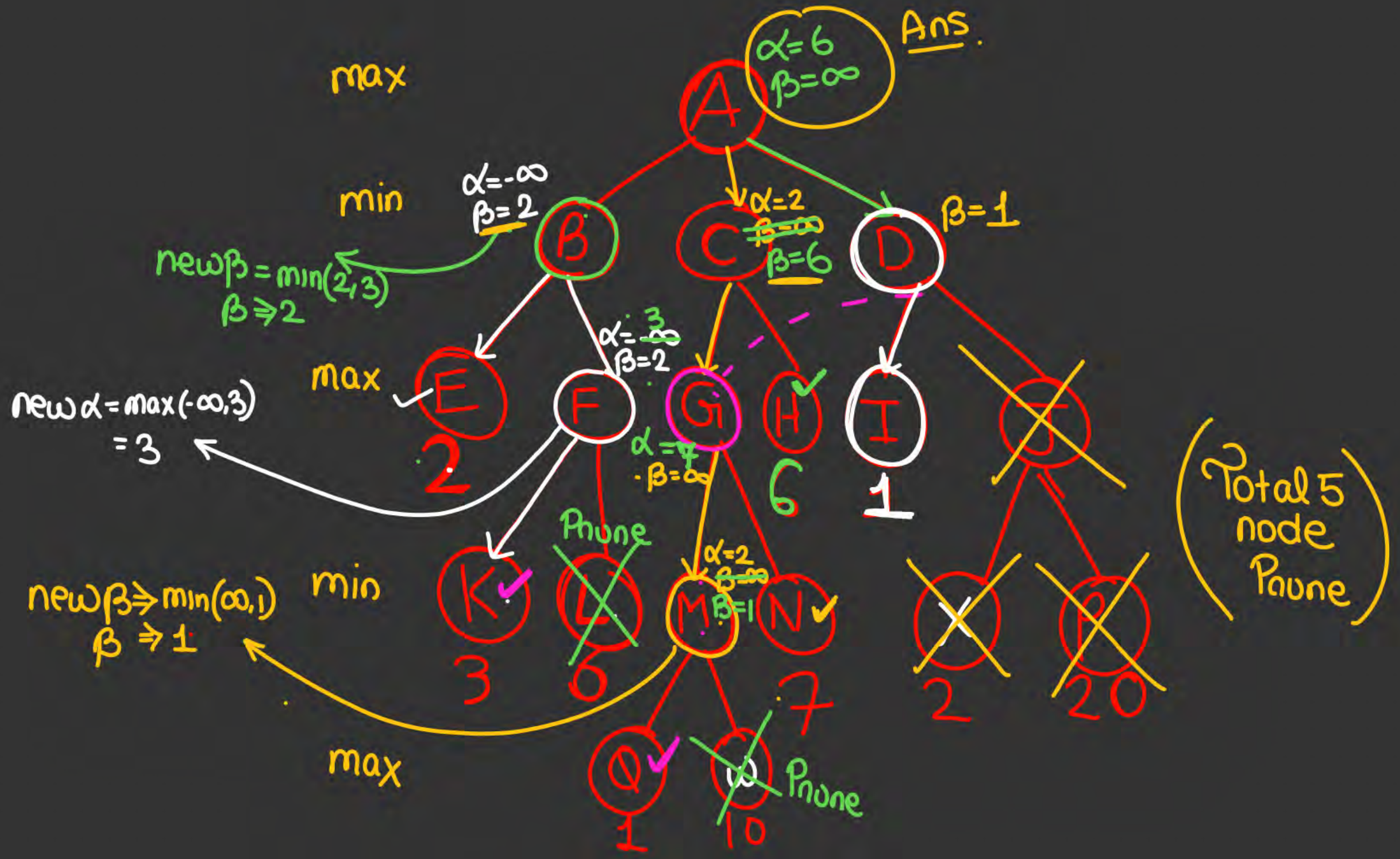
(D) No nodes are pruned


max
min
max
min
max
α=-∞
β=∞
A
α=-∞
β=∞
β=min(∞,2)
=2
B
C
D
E
2
F
G
H
6
I
1
J
K
3
L
6
M
N
7
X
2
P
20
Q
1
W
10


max
A
α=-∞
β=∞
new α = max(-∞,2)
=2
min
B
α=-∞
β=2
new β = min(2,3)
β ⇒ 2
C
α=2
β=∞
D
@G α ⇒ max(2,1)
α ⇒ 2
β=∞
No Prune
Now move to N
new α = max(2,7)
⇒ 7
max
E
2
F
α=-∞ 3
β=2
new α = max(-∞,3)
=3
G
α=2
β=∞
H
6
I
1
J
K
3
L
6
Prune
min
new β ⇒ min(∞,1)
β ⇒ 1
M
α=2
β=∞
β=1
N
7
X
2
P
20
max
Q
1
∞
10
Prune


max
min
max
min
max
α=-∞
β=∞
A
new α = max(-∞,2)
=2
@C new β= min(∞,7)
(β=7
α=2)
Now move to
H⇒β=6
α=-∞
β=2
B
new β = min(2,3)
β⇒2
α=2
β=6
C
D
α=3
β=2
F
E
2
new α = max(-∞,3)
=3
G
α=7
β=∞
H
6
I
1
J
Prune
K
3
L
6
α=2
β=1
M
N
7
X
2
P
20
new β⇒min(∞,1)
β ⇒1
Q
1
∞
10
Prune


max
A
α=6
β=∞
new α= max(2,6)
= 6
min
B
α=-∞
β=2
C
α=2
β=6
D
α=6
β=∞
new β = min(2,3)
β ⇒ 2
new β ⇒ min(∞,1)
β= 1
So α=6
β= 1
Prune
max
E
2
F
α= 3
β=2
G
α=4
β=∞
H
6
I
1
J
new α = max(-∞,3)
= 3
Prune
K
3
L
6
M
α=2
β=1
N
7
O
2
P
20
new β ⇒ min(∞,1)
β ⇒ 1
min
max
Q
1
R
10
Prune
Total 5
node
Prune


max
min
max
min
max
A
α=6
β=∞
Ans.
B
α=-∞
β=2
C
α=2
β=6
D
β=1
newβ = min(2,3)
β ⇒ 2
E
2
F
α=3
β=2
G
α=4
β=∞
H
6
I
1
J
newα = max(-∞,3)
=3
K
3
L
6
Prune
M
α=2
β=1
N
7
O
1
P
10
Prune
Q
2
R
20
newβ ⇒ min(∞,1)
β ⇒ 1
(Total 5 node Prune)

Alpha-Beta Pruning – The complete procedure

- The procedure for solving any general alpha-beta pruning problem can be broken down into a set of steps that are followed to systematically explore the game tree while minimizing the number of evaluations. Below is the step-by-step procedure for solving any alpha-beta pruning problem:
- 1. Initialization:
  - Start at the root of the game tree.
  - Set initial values for Alpha to $-\infty$ (lowest possible) and Beta to ∞ (highest possible).
  - Alpha: Keeps track of the maximum lower bound that the maximizing player (MAX) is assured of.
  - Beta: Keeps track of the minimum upper bound that the minimizing player (MIN) is assured of.

Alpha-Beta Pruning – The complete procedure

- 2. Traverse the Tree Depth-First: Use a depth-first search (DFS) traversal to explore the tree. Start from the root node and proceed down one branch, only moving to another branch if necessary.
- 3. Update Alpha and Beta:
  - At each node, based on whether it is a MAX or MIN node:
  - MAX node:
  - Maximizing player (goal is to maximize the utility value).
  - Start with the current value of Alpha.
  - At each child, update Alpha with the maximum value of its children.
  - Update: Alpha = max(Alpha, child value).
  - MIN node:
  - Minimizing player (goal is to minimize the utility value).
  - Start with the current value of Beta.
  - At each child, update Beta with the minimum value of its children.
  - Update: Beta = min(Beta, child value).

Alpha-Beta Pruning – The complete procedure

- 4. Prune When Alpha ≥ Beta:
- While traversing the tree, check if Alpha ≥ Beta at any point: Alpha ≥ Beta means that the current branch will not affect the outcome because the maximizing player can already guarantee a better result or the minimizing player can avoid a worse outcome.
- When this condition holds, you can prune the remaining branches below that node, as further exploration is unnecessary.
- 5. Backtrack and Propagate Values:
- Once a leaf node is evaluated, propagate the computed value up to its parent node:
- At MAX nodes, propagate the maximum value from the children.
- At MIN nodes, propagate the minimum value from the children.

Alpha-Beta Pruning – The complete procedure

- 6. Repeat Until All Nodes Are Visited or Pruned:
- Continue traversing the tree until: You have visited all necessary nodes, or Pruning has eliminated further exploration.

THANK - YOU